ASSIGNMENT 3
PRATHAM SHARMA 2401010229

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following—

1. **Create the tables with the appropriate integrity constraints.**

```sql
CREATE TABLE Customer (
    cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50) NOT NULL,
    price INT CHECK (price > 0)
);

CREATE TABLE Sale (
    bill_no INT,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT CHECK (qty_sold > 0),
    PRIMARY KEY (bill_no, item_id),
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2. Insert around 10 records in each of the tables.

```
INSERT INTO Item VALUES
(101,'Pen',20), (102,'Notebook',50), (103,'Bag',500),
(104,'Pencil',10), (105,'Shoes',1200),
(106,'Bottle',150), (107,'Watch',800),
(108,'Book',300), (109,'Headphones',700), (110,'Mouse',250);
```

```
INSERT INTO Customer VALUES
(1,'Amit'), (2,'Riya'), (3,'Rahul'), (4,'Sneha'), (5,'Karan'),
(6,'Neha'), (7,'Vikas'), (8,'Pooja'), (9,'Arjun'), (10,'Meera');
```

```
INSERT INTO Sale VALUES
(1,CURDATE(),1,101,2),
(1,CURDATE(),1,103,1),
(2,CURDATE(),2,105,1),
(3,CURDATE(),3,108,2),
(4,CURDATE(),4,102,5),
(5,CURDATE(),5,110,1),
(6,CURDATE(),6,107,1),
(7,CURDATE(),7,106,3),
(8,CURDATE(),8,104,10),
(9,CURDATE(),9,109,2);
```

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT s.bill_no, s.item_id, s.qty_sold, i.price,
       (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.*
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

6. Give a count of how many products have been bought by each customer

```
SELECT c.cust_id, c.cust_name, SUM(s.qty_sold) AS total_products
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE VIEW Bill_View AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id,
       i.price, s.qty_sold,
       (i.price * s.qty_sold) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE VIEW Weekly_Sales AS
SELECT bill_date,
       SUM(i.price * s.qty_sold) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;
```

OUTPUT :

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	bill_no	cust_name	item_id
1	1	Ishaan	101
2	1	Ishaan	103
3	2	Mahin	105
4	3	Aditya	108
5	4	Ayush	102
6	5	Priya	110
7	6	Karan	107
8	7	Neha	106
9	8	Rohit	104
10	9	Simran	109

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	bill_no	item_id	qty_sold	price	amount
1	1	101	2	50000	100000
2	1001	101	1	50000	50000
3	4	102	5	500	2500
4	1002	102	2	500	1000
5	1	103	1	800	800
6	1007	103	2	800	1600
7	8	104	10	12000	120000
8	1003	104	1	12000	12000
9	2	105	1	200	200
10	1004	105	3	200	600
11	7	106	3	1500	4500
12	1009	106	1	1500	1500
13	6	107	1	900	900
14	1010	107	2	900	1800
15	3	108	2	25000	50000
16	1005	108	1	25000	25000
17	9	109	2	30000	60000
18	1008	109	1	30000	30000
19	5	110	1	700	700

Log | 1: Sale [10] | 2: Sale [20] | **3: Customer [10]** | 4: Customer [10] | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	cust_id	cust_name
1	1	Ishaan
2	4	Ayush
3	2	Mahin
4	7	Neha
5	8	Rohit
6	3	Aditya
7	9	Simran
8	6	Karan
9	10	Vikas
10	5	Priya

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | **4: Customer [10]** | 5: Sale [2] | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	cust_id	cust_name	total_products
1	1	Ishaan	4
2	2	Mahin	3
3	3	Aditya	3
4	4	Ayush	8
5	5	Priya	2
6	6	Karan	5
7	7	Neha	5
8	8	Rohit	11
9	9	Simran	3
10	10	Vikas	2

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | **5: Sale [2]** | 6: Item [10]

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	item_name
1	Pen Drive
2	Tablet

Log | 1: Sale [10] | 2: Sale [20] | 3: Customer [10] | 4: Customer [10] | 5: Sale [2] | **6: Item [10]**

In **DbVisualizer Pro** this feature includes data editing functionality, management of binary/BLOB data, and more...

*	item_id	item_name	price
1	101	Keyboard	50000
2	103	Keyboard	800
3	105	USB Cable	200
4	108	Tablet	25000
5	102	Mouse	500
6	110	Pen Drive	700
7	107	Charger	900
8	106	Headphones	1500
9	104	Monitor	12000
10	109	Smartphone	30000